

RESEARCH ARTICLE

Submission Under Consideration at Ledger

MOSAIC: State-Transition-Centric Integrity Capsules for Distributed Ledger Execution

Adnan Omar Awad Allemon,*

Abstract. Distributed ledgers commonly make a globally ordered block or log the primary integrity boundary, even when application transitions touch disjoint resources. This paper introduces *Evidence-Complete Transition Closure* (ECTC), a protocol semantics in which a state transition is not terminal merely because a validator accepted it. A Capsule must either reach a Closed outcome carrying a ClosureProof and successor StateSeal, a Conflict outcome carrying independently verifiable ConflictEvidence, or an Abandoned outcome carrying an AbandonProof. The protocol therefore binds the predecessor state, transition claim, witness obligation, and successor state, while preserving incompatible and abandoned claims as auditable artifacts. MOSAIC (Mutual-Obligation State And Integrity Capsules) is an executable prototype of this lifecycle. It includes Ed25519 signatures, leaderless TCP dissemination, SQLite WAL recovery, weighted membership and settlement, commit–reveal randomness, Reed–Solomon availability, deterministic execution, onboarding, and a hash-chained event log, but these components are treated as implementation support rather than individual novelty claims. We state an unweighted quorum-intersection argument for closure exclusivity, describe a weighted extension, map eight paper invariants to a bounded TLA+ model, and add adversarial state-machine tests. In a 1,000-operation serialized local workload, disjoint transitions produced 3.54/3.63 ms p50/p95 latency, while a genuine same-resource competing-claims workload produced 4.19/5.02 ms p50/p95 latency, rejected 1,000 losing claims, and generated 1,000 ConflictEvidence artifacts. Batch sizes of 10 and 100 reduced measured protocol bytes per successful operation to 2,530.78 and 2,473.96 bytes. A seven-process local long-run with two Byzantine test identities, a kill/restart event, and partial-frame probes completed all 120 operations with liveness 1.0, zero unexpected operational errors, p50 36.80 ms, and p95 233.56 ms. These are LOCAL EMULATION observations, not WAN or public-testnet evidence. The contribution is a reproducible protocol-semantics hypothesis and systems artifact, not a claim of universal blockchain superiority.

KEY WORDS

1. Evidence-complete transition closure.
2. State-transition integrity.
3. Distributed-ledger protocol.
4. Byzantine fault tolerance.
5. Conflict evidence.
6. Deterministic execution.
7. Reproducible systems.

* A. O. A. Allemon (adnanomar774@gmail.com) is an independent researcher.

1. Introduction

Blockchains are often introduced through the abstraction of a chain of blocks.¹ The engineering problem is more specific: independent machines must agree on which state transitions are valid, how competing claims are handled, what state follows an accepted transition, and how a later observer can verify both accepted and rejected claims. A global order is a powerful general mechanism, but it also places unrelated activity through common ordering and commit paths. Recent systems have therefore decomposed dissemination, ordering, execution, and certification in different ways. HotStuff provides a partially synchronous Byzantine fault-tolerant replication protocol with a formal safety argument and responsive communication phases;² Narwhal and Tusk separate reliable dissemination from BFT ordering;³ FastPay uses Byzantine consistent broadcast for a restricted pre-funded settlement workload;⁴ and Sui Lutriss combines a consensusless path with consensus when concurrent access patterns create an ordering risk.⁵ These systems establish that a block is not the only possible abstraction around which a ledger can organize agreement.

MOSAIC studies a narrower question: can the *state transition itself* be made the integrity boundary, with a verifiable terminal artifact for every observed outcome? Its basic object, a Capsule, names a predecessor StateSeal, a deterministic operation, an attempt and epoch context, and witness obligations. A validator may issue a signed WitnessReceipt, but a receipt alone does not mutate state. A first-claim lock prevents a validator from acknowledging incompatible claims for the same resource and protocol context. A ClosureProof binds compatible receipts to the Capsule and its predecessor. Deterministic execution computes a successor root, which becomes a StateSeal. If a competing claim is observed, ConflictEvidence records it; if a claim cannot close under its obligations, an AbandonProof records that outcome. The proposed semantics are called Evidence-Complete Transition Closure (ECTC).

ECTC is intentionally not defined by the use of Ed25519, SQLite, Reed–Solomon coding, gossip, quorum voting, deterministic execution, commit–reveal randomness, or stake settlement. Those are known components. The proposed contribution is the lifecycle and its integrity semantics: a transition is not silently lost between acceptance and application, and a negative outcome is represented by evidence rather than by an unexplained absence. This claim is adjacent to object-centric execution, optimistic concurrency, authenticated state, and BFT certificates. Tango studied distributed data structures over a shared log;⁶ Block-STM studied deterministic optimistic parallel execution and dependency validation;⁷ and process-commutative distributed objects studied the space between BFT CRDTs and totally ordered ledgers.⁸ MOSAIC therefore does not claim to be the first object-centric, conflict-aware, or selectively ordered ledger.

The paper makes three contributions. First, it defines ECTC as a protocol abstraction and gives executable representations for Capsule, WitnessReceipt, first-claim lock, ClosureProof, StateSeal, ConflictEvidence, and AbandonProof. Second, it gives eight protocol invariants and a quorum-intersection argument for closure exclusivity, while connecting the argument to a bounded TLA+ model and explicitly stating what remains only modeled, implemented, or observed. Third, it provides a reproducible implementation and evaluation: an adversarial state-machine matrix, a 1,000-operation disjoint-versus-contended workload, a batching study, a 4/7/10/16-validator local scaling sweep, a detailed byte decomposition, and a seven-process restart rehearsal.

The paper is organized as follows. Section 2 defines the system model and problem. Section

3 presents the MOSAIC protocol. Section 4 defines ECTC, invariants, quorum assumptions, and closure exclusivity. Section 5 describes the implementation. Section 6 reports formal and adversarial validation. Section 7 evaluates workloads, scaling, batching, recovery, and communication overhead. Section 8 discusses related work. Section 9 analyzes security and limitations. Sections 10–12 cover reproducibility, discussion, and conclusion.

2. System Model and Problem Definition

Let R be a set of logical resources, such as an account, object, contract slot, or application-specific state domain. Each resource $r \in R$ has a current state root and a StateSeal identifying the state that an honest implementation believes to be current. A protocol context is the pair (r, e) consisting of a resource and an epoch. A transition may be independent of another transition when their resource contexts are disjoint; this does not mean that MOSAIC eliminates ordering, but that it makes the scope of ordering explicit at the resource boundary rather than assuming that every transition requires the same global ordering path.

The validator set is V . Model A is unweighted: $N = |V|$, at most f validators are Byzantine, $N \geq 3f + 1$, and a closure quorum has at least $2f + 1$ validators. Model B is weighted: every validator v has positive weight $w(v)$, total weight is W , Byzantine weight is at most $W/3$, and a quorum has weight strictly greater than $2W/3$. The implementation supports weighted thresholds; the principal theorem is stated first for Model A so that the assumptions are not hidden inside a single overloaded definition.

The adversary may submit malformed, replayed, stale, conflicting, or delayed messages; may cause a validator to fail and restart; may withhold a beacon reveal; and may cause shard loss in the local availability layer. The model does not assume protection against compromised private keys, a compromised host operating system, supply-chain compromise, an economically dominant stake coalition, or network partitions beyond the declared BFT assumptions. The evaluation uses deterministic local adversarial schedules and does not substitute one host for independent WAN operators.

The problem is to make the following four relations explicit and auditable:

- (1) a transition claim must bind to a known predecessor state;
- (2) witness obligations must be bound to the same claim and protocol context;
- (3) a successful transition must bind the closure certificate to the successor state;
- (4) a competing or abandoned claim must have a terminal evidence artifact rather than disappearing from the observable protocol history.

3. MOSAIC Protocol

3.1. *Core objects*—A Capsule is modeled as

$$C = (r, p, o, n, w, e, \sigma), \quad (1)$$

where r is the resource identifier, p the predecessor StateSeal identifier, o the deterministic operation, n the attempt/nonce context, w the witness rule, e the protocol epoch, and σ the originator signature. The Capsule is a signed claim that a transition may be attempted from p ; it is not itself a state mutation.

A WitnessReceipt is a signed statement by validator v about a Capsule. It binds the Capsule

identifier, predecessor identifier, witness identifier, epoch, attempt, status, and signature. An ACCEPT receipt is eligible for closure only when its signature and context verify and the validator has not already locked an incompatible claim. An ABANDON receipt contributes to an AbandonProof when the abandonment rules permit it.

A first-claim lock $L(r, v)$ is initially empty. A validator may set the lock to one Capsule for the resource context. Repeated observations of the same Capsule and attempt are idempotent; an incompatible claim produces ConflictEvidence. The lock is local state, but a quorum of compatible receipts makes the local non-equivocation assumption relevant to closure safety.

A ClosureProof contains one Capsule identifier, its predecessor, epoch and attempt, the ordered set of compatible WitnessReceipts, the signer identifiers, and a proof identifier. Closure is accepted only if the proof verifies, its quorum threshold is met, and no incompatible evidence for the same predecessor has been recorded. A StateSeal binds the successor state root, resource, version, owner/capability context, and transition digest. Applying a ClosureProof computes and persists the successor StateSeal atomically with the implementation's state transition.

ConflictEvidence contains two distinct Capsule identifiers sharing a predecessor and their originator signatures. AbandonProof contains the Capsule, predecessor, epoch, attempt, signer identifiers and ABANDON receipts. BundleClosure composes closures for independent resources and applies them at one visibility boundary; it is auxiliary to the ECTC contribution.

Table 1. MOSAIC objects and their ECTC roles.

Object	Role in the protocol
StateSeal	Authenticated successor-state digest and predecessor reference for the next transition.
Capsule	Signed claim naming resource, predecessor, operation, attempt, epoch, and witness rule.
WitnessReceipt	Signed validator acknowledgement tied to one Capsule and context.
ClosureProof	Quorum-backed certificate that a Capsule may be applied.
ConflictEvidence	Independently verifiable record of incompatible claims for one predecessor context.
AbandonProof	Evidence that a claim ended without closure under the applicable obligation context.
BundleClosure	Optional composition of several independent ClosureProofs.

3.2. Closure algorithm—The following pseudocode states the normal lifecycle without conflating receipt, closure, execution, and persistence.

Listing 1. Capsule Closure Algorithm.

```

Input: Capsule C, predecessor seal P
1. Verify signature(C), epoch(C), nonce(C), and predecessor(P).
2. If P is not restored, enqueue C as pending; do not classify it as daemon failure.
3. Acquire first-claim lock L(resource(C), validator).
4. If an incompatible lock exists, emit ConflictEvidence(C, incumbent).
5. Collect valid ACCEPT WitnessReceipts for C.
6. If quorum Q is not satisfied, retain C as pending or create AbandonProof.
7. Construct and verify ClosureProof(C, Q).
8. Execute C deterministically from P.
9. Compute successor root S' and construct StateSeal(S').
10. Persist closure, successor seal, and audit event atomically.
11. Return Closed(C, ClosureProof, StateSeal).

```

Listing 2. Conflict handling.

```

Input: Capsule C and incumbent claim I for the same predecessor

```

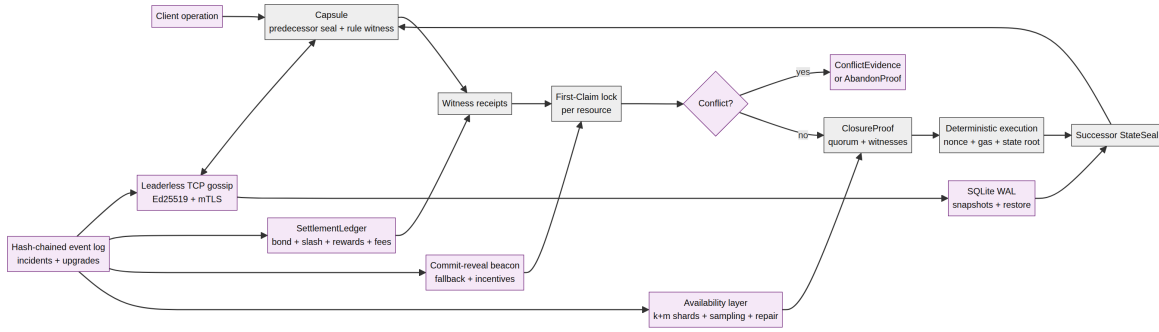


Fig. 1. MOSAIC architecture. The ECTC lifecycle is the protocol core; settlement, randomness, availability, networking, durable storage, and event logging support that lifecycle rather than constituting independent novelty claims.

1. Verify that C and I are distinct and share the same lock context.
2. Preserve both claim identifiers and originator signatures.
3. Construct $\text{ConflictEvidence}(C, I)$.
4. Reject a ClosureProof that omits the conflict or tries to close both claims.
5. Return $\text{Conflict}(C, \text{ConflictEvidence})$.

Listing 3. Recovery handling.

- Input: incoming Capsule C and durable store W
1. Restore known and current StateSeals, capsules, receipts, and closures from W .
 2. If $\text{predecessor}(C)$ is absent, place C in a bounded pending queue.
 3. When $\text{predecessor}(C)$ becomes known, re-run validation and receipt processing.
 4. If a valid closure and successor seal are restored, expose $\text{Closed}(C)$.
 5. Otherwise preserve ConflictEvidence or AbandonProof if present.
 6. Append a hash-chained recovery event.

4. Evidence-Complete Transition Closure

4.1. *Definition*—For every Capsule C , define the observed terminal outcome as

$$\text{Outcome}(C) \in \{\text{Closed}, \text{Conflict}, \text{Abandoned}\}. \quad (2)$$

The Closed outcome requires a valid ClosureProof and a successor StateSeal . The Conflict outcome requires ConflictEvidence naming the incompatible claims. The Abandoned outcome requires an AbandonProof . If none of these artifacts has been observed, the local state is *pending*, not a terminal outcome; the protocol must not silently infer success or failure.

The implementation exposes this definition through the ECTCStatus and TransitionOutcome types. `MosaicProtocol.transition_outcome` returns Closed only after both closure and successor seal are present, returns Conflict when evidence names the Capsule, returns Abandoned when the abandonment proof exists, and otherwise returns no terminal outcome. This distinction makes the semantics testable without claiming that every network participant has instantaneous knowledge of every artifact.

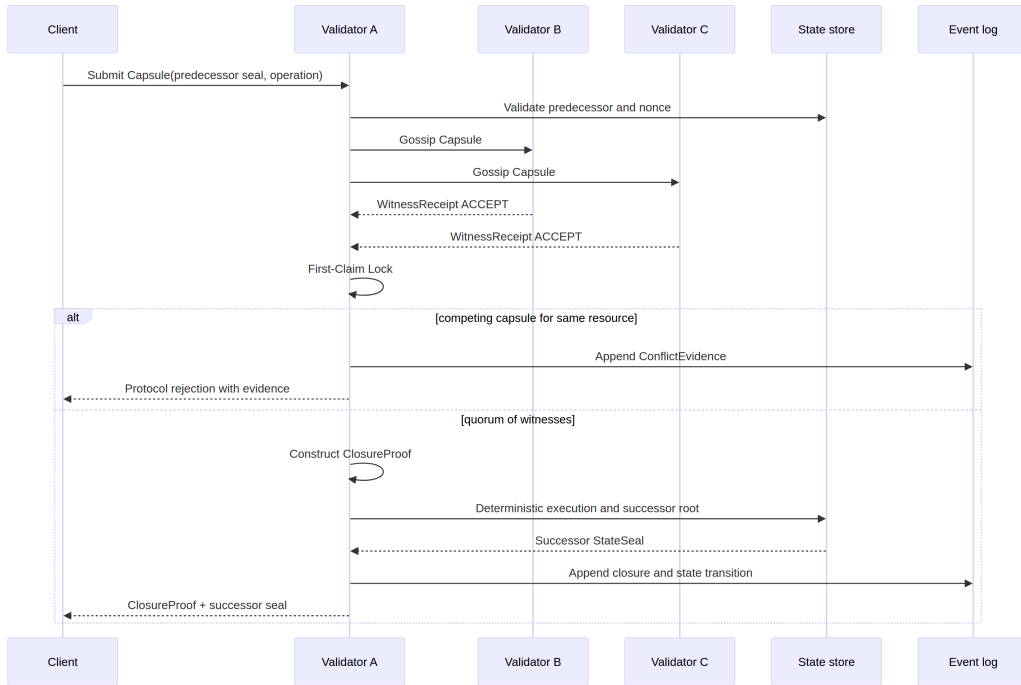


Fig. 2. Capsule lifecycle with successful and conflicting paths. A valid Capsule receives compatible receipts and either reaches closure or produces explicit evidence.

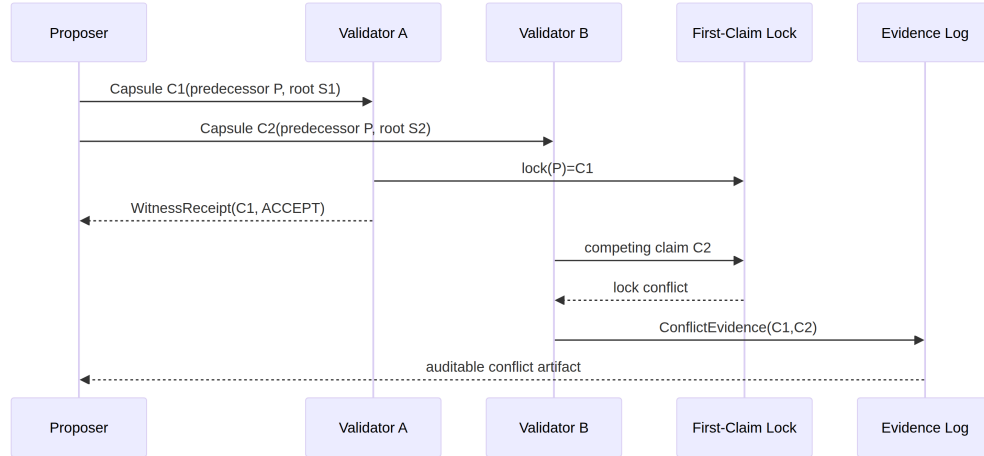


Fig. 3. Two competing Capsules sharing one predecessor. The first-claim lock prevents an honest validator from acknowledging both; the losing path is retained as ConflictEvidence.

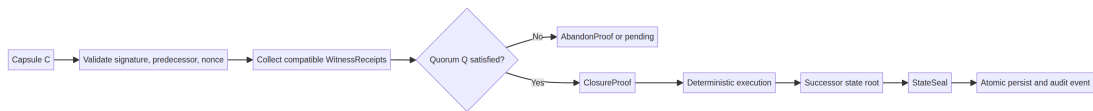


Fig. 4. Normal closure path: Capsule validation, compatible receipts, quorum, ClosureProof, deterministic execution, successor root, StateSeal, and atomic audit persistence.

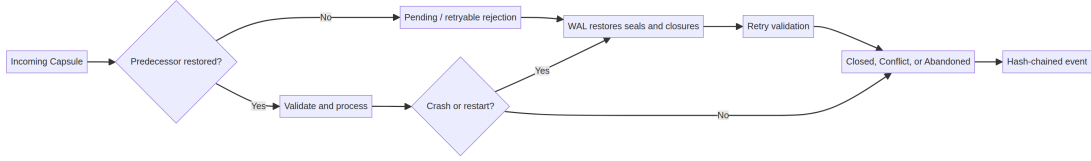


Fig. 5. Failure and recovery path. An unknown predecessor is pending until WAL restoration makes the seal available; recovery is observable in the event log rather than counted as an unexplained daemon failure.

4.2. *Protocol invariants*—The paper invariants are stated below. The TLA+ file `formal/mosaic_safety.tla` uses the same names; the bounded model maps finite-state transitions to these properties.

- $I1 : \text{Apply}(C) \Rightarrow \text{ValidClosure}(C),$
- $I2 : \text{Closure}(C) \Rightarrow \text{pred}(C) = \text{currentState},$
- $I3 : \text{StateSeal}(C) \Rightarrow H(C, p, s', Q) = \text{Seal},$
- $I4 : \text{Closed}(C_1, r) \wedge \text{Closed}(C_2, r) \Rightarrow \text{Compatible}(C_1, C_2),$
- $I5 : L(r, v) = C_1 \Rightarrow v \text{ does not sign incompatible } C_2,$
- $I6 : \text{Conflict}(C_1, C_2) \Rightarrow \text{ConflictEvidence}(C_1, C_2),$
- $I7 : \text{Abandon}(C) \Rightarrow \text{AbandonProof}(C),$
- $I8 : \text{Execute}(C, s) = s' \text{ is deterministic for honest validators.}$

I1–I3 bind application and state seals to closure data. I4 and I5 are the safety boundary. I6 and I7 state accountability for negative outcomes. I8 is restricted to the supported deterministic instruction kernel, not an arbitrary smart-contract language.

4.3. *Quorum model and closure exclusivity*—In Model A, let $N \geq 3f + 1$ and let each valid closure quorum have cardinality at least $2f + 1$. For any two quorums Q_1, Q_2 ,

$$|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - N \geq 4f + 2 - (3f + 1) = f + 1. \quad (3)$$

At most f validators are Byzantine, so $Q_1 \cap Q_2$ contains an honest validator.

Theorem 1 (Closure Exclusivity). Under the Model A quorum-intersection and honest non-equivocation assumptions, two incompatible Capsules cannot both obtain valid ClosureProofs for the same resource and protocol context.

Proof. Assume for contradiction that incompatible Capsules C_1 and C_2 both obtain valid ClosureProofs with signer quorums Q_1 and Q_2 . The intersection argument above gives an honest validator $h \in Q_1 \cap Q_2$. Because both proofs are valid, h issued an ACCEPT receipt for both Capsules. Since the Capsules are incompatible in the same first-claim context, this violates the honest non-equivocation assumption enforced by the first-claim lock. Therefore both ClosureProofs cannot be valid simultaneously. $\square$

The theorem is conditional. It does not prove liveness, cryptographic key security, permissionless Sybil resistance, economic incentive compatibility, or correctness under arbitrary reconfiguration. In Model B, if $w(Q_i) > 2W/3$ and Byzantine weight is at most $W/3$, then $w(Q_1 \cap Q_2) > W/3$ and the intersection contains positive honest weight. The executable `mosaic/safety.py` and model checker validate finite instances of these premises; they do not replace the theorem or establish an unbounded formal proof.

5. Implementation

The prototype is written for Python 3.11 or newer. The public repository contains a `mosaic` namespace for the protocol and permissionless layers and a `ccd_nexus` namespace for shared cryptographic primitives and legacy support. The code uses canonical serialization, Ed25519 signatures, digest-bound identifiers, and typed dataclasses for protocol objects.

5.1. Networking and storage—The network layer uses asyncio TCP with a four-byte length prefix and JSON payload framing. It supports optional mutual TLS, replay protection, per-peer token buckets, connection caps, frame-size limits, and frame timeouts. Invalid or oversized frames are classified as malformed protocol input; unexpected structural exceptions remain daemon errors. The leaderless dissemination layer does not require a single permanent coordinator for every Capsule.

Durability uses SQLite WAL with full synchronous mode, schema migration, snapshots, and restore of seals, Capsules, receipts, and closures. The long-run harness exposed a race in which a restarted node could receive a Capsule before predecessor seals had completed restoration. The corrected path keeps the Capsule pending or records a retryable protocol rejection. The recovery artifact now reports zero unexpected errors in the v5 local run.

5.2. Settlement and randomness—Membership includes signed admission requests, stake bonds, weighted committee selection, withdrawal delay, and slash evidence. SettlementLedger persists bonds, unbonding, rewards, penalties, and fees. This is an executable accounting component, not an externally enforced public economy. The randomness layer uses commit-reveal rounds with fallback and reward/penalty settlement. It provides an operational mechanism for non-reveal handling, not a proof of unbiased randomness or incentive compatibility against all strategic behavior.

5.3. Availability and execution—The availability layer uses Reed–Solomon coding over GF(256) with data and parity shards.⁹ It exposes shard storage, sampling, repair, and WAL persistence. The availability network has PUT, FETCH, SAMPLE, and REPAIR paths, but the current evidence is local and does not establish a globally deployed availability network.

The execution kernel supports deterministic SET, DELETE, and ADD_INT-style instructions with origin, nonce, gas, and state-root binding. Execution is atomic and returns a receipt containing pre- and post-state roots. The kernel is a vehicle for studying binding between protocol closure and execution; it is not a general smart-contract VM.

6. Formal and Adversarial Validation

6.1. TLA+ mapping—The TLA+ model names the following properties: I1_NoApplyWithoutClosure, I2_PredecessorBinding, I3_SealBinding, I4_NoTwoIncompatibleClosures, I5_FirstClaimNonEquivalent, I6_ConflictEvidencePreserved, I7_AbandonEvidencePreserved, and I8_DeterministicExecution. The model abstracts signatures and execution details and checks finite sets only. The Python model checker separately enumerates small unweighted committees and checks weighted concrete instances. In the final run, 84 finite committee cases and 36,703 quorum pairs were checked for the unweighted intersection property; the weighted extension used a concrete four-validator weight assignment and passed its stated premise check.

6.2. Adversarial matrix—Table 2 summarizes the required negative and recovery paths. The entries are executable tests or bounded checks, not claims that a single test proves a distributed

protocol theorem.

Table 2. Adversarial state-machine matrix.

No.	Scenario	Evidence location and scope
1	Two conflicting Capsules	ConflictEvidence tests; local first-claim lock.
2	Duplicate Capsule	Identifier-boundary idempotence test.
3	Replay old WitnessReceipt	Epoch replay rejection test.
4	Validator signs competing Capsule	Competing-successor lock test.
5	Invalid predecessor	Forged predecessor test.
6	Invalid nonce	Deterministic execution nonce test.
7	Invalid successor root	Empty-root and closure-binding tests.
8	Insufficient quorum	Closure threshold tests.
9	Conflicting quorum	Honest-intersection model and invariant test.
10	Restart before predecessor restoration	Long-run artifact and durable restore test.
11	Restart after closure	Durable close/restart test.
12	Partial frame	Frame-timeout test.
13	Oversized frame	Malformed-frame rejection test.
14	Replay after restart	ReplayGuard and epoch receipt tests.
15	Delayed receipt	Old-attempt receipt test.
16	Conflicting receipts	ClosureProof consistency test.
17	Byzantine non-reveal	Beacon fallback test.
18	Availability shard loss	Reed–Solomon loss/repair tests.
19	Repair after restart	Availability-store restore/repair test.
20	Independent resources	Independent-resource and BundleClosure tests.

6.3. Verification gates—The final repository collects 102 automated tests. The deterministic wire-fuzz harness exercises 4,000 parser cases. The bounded model checker reports twelve result records, including unweighted and weighted quorum checks, ECTC outcomes, bundle atomicity, availability certificate checks, and execution atomicity. These counts describe the current repository state and are not independent peer review or a formal proof.

7. Experimental Evaluation

7.1. Methodology and scope—The evaluation uses two complementary harnesses. The first is an in-process serialized workload that executes real Capsules, receipts, closures, seals, and hash-chained event records. It compares 1,000 disjoint transitions against 1,000 same-resource competing-claims iterations, evaluates BundleClosure batch sizes 1, 10, and 100 on independent resources, and runs a 120-operation disjoint scaling sweep at 4, 7, 10, and 16 validator processes. The second is the seven-process long-run daemon rehearsal with two Byzantine test identities, a scheduled kill/restart, and partial-frame probes. Neither harness is a WAN experiment.

7.2. Disjoint, contended, and batching workloads—The disjoint workload creates 1,000 independent resources and closes one transition on each. The same-resource competing-claims workload creates two distinct Capsules from the same predecessor before either claim is closed.

The first claim obtains a ClosureProof and successor StateSeal; the second claim is submitted to the same first-claim context and must be rejected with ConflictEvidence. Each of the 1,000 iterations therefore has one successful closure, one rejected claim, and one evidence artifact. Both the disjoint and competing workloads use four validator objects and four ACCEPT receipts for the winning closure. The batching workload creates independent closures and applies them through BundleClosure at batch sizes 1, 10, and 100. All event logs verified successfully.

Table 3. ECTC workload measurements from 1,000 operations.

Workload	p50 ms	p95 ms	throughput/s	bytes/op	conflicts
Disjoint	3.54	3.63	258.27	3,071.22	0
Competing claims	4.19	5.02	207.79	4,945.95	1,000
Batch 1	8.23	14.60	87.39	3,096.15	0
Batch 10	8.78	15.07	85.55	2,530.78	0
Batch 100	8.39	14.38	89.03	2,473.96	0

The competing-claims row is not a serial same-resource control: two claims are created before the winner is closed, and the loser follows the first-claim conflict path. Every iteration produced one Closed winner, one rejected loser, and one ConflictEvidence artifact; the event log remained verifiable. The measured evidence payload totaled 635,000 bytes, or 635 bytes per rejected claim, and the complete competing-claims accounting was 4,945.97 bytes per successful closure. This contrast supports the state-transition-centric claim in a limited but direct way: independent transitions avoid conflict evidence, whereas incompatible claims sharing one predecessor produce an explicit, measurable terminal artifact. Batch 100 reduces measured protocol bytes per operation by approximately 19.4% relative to the disjoint baseline in this implementation and workload, while its p95 remains close to batch 10. These are implementation measurements, not claims of superiority over another ledger or of simultaneous WAN concurrency.

7.3. Scaling sweep—The local disjoint sweep used 120 operations and varied validator process count. Throughput decreased from 257.31 operations/s at four validators to 156.45 at seven, 112.16 at ten, and 71.87 at sixteen. The corresponding p50 latencies were 3.54, 6.01, 8.47, and 13.37 ms. The curve is a measurement of the in-process Python implementation and its receipt fan-out, not a consensus lower bound.

Table 4. Local validator-process scaling sweep, 120 disjoint operations.

Validators	p50 ms	p95 ms	throughput/s	bytes/op
4	3.54	3.69	257.31	3,065.70
7	6.01	6.11	156.45	4,073.62
10	8.47	8.62	112.16	5,081.67
16	13.37	13.56	71.87	7,109.56

7.4. Protocol and implementation overhead—The byte accounting uses actual JSON wire encodings and actual event-log records. For the disjoint 1,000-operation workload, the components are shown in Table 5. ClosureProof wrapper excludes nested receipt payloads so that receipts are not double-counted; Closure wire is reported separately for inspection. ConflictEvidence is zero in the disjoint row but is measured separately in the competing-claims row. The protocol-bytes total sums Capsule, WitnessReceipt, ClosureProof wrapper, StateSeal, four-byte framing overhead, event-log records, ConflictEvidence, availability bytes, and bundle

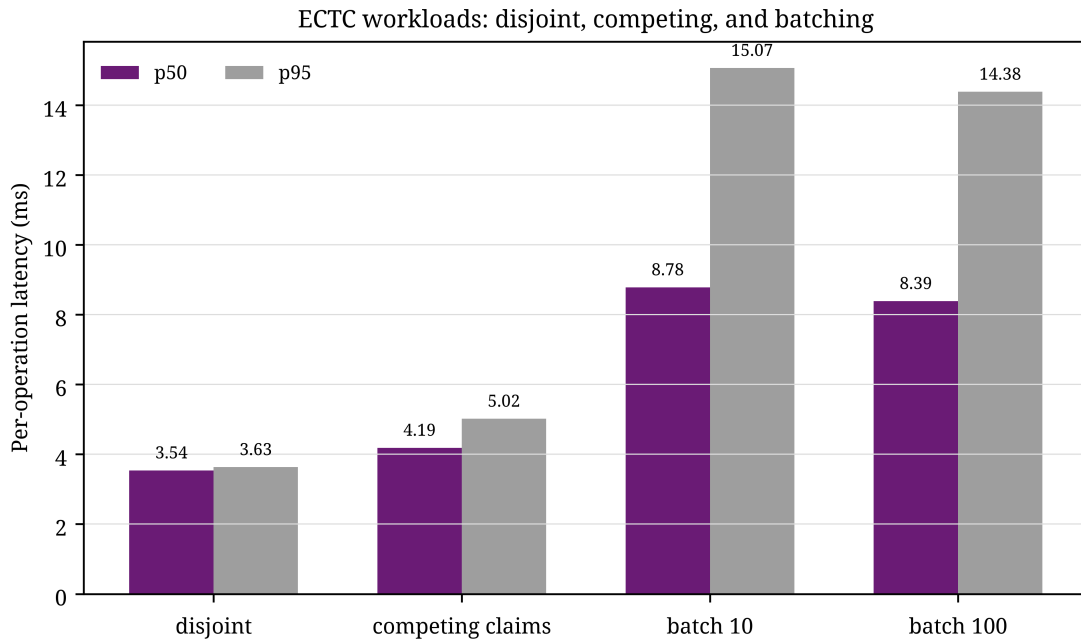


Fig. 6. Measured p50 and p95 latency for disjoint, competing-claim, and BundleClosure workloads. The competing workload creates two claims from one predecessor before closing the winner; the local design still does not model WAN concurrency.

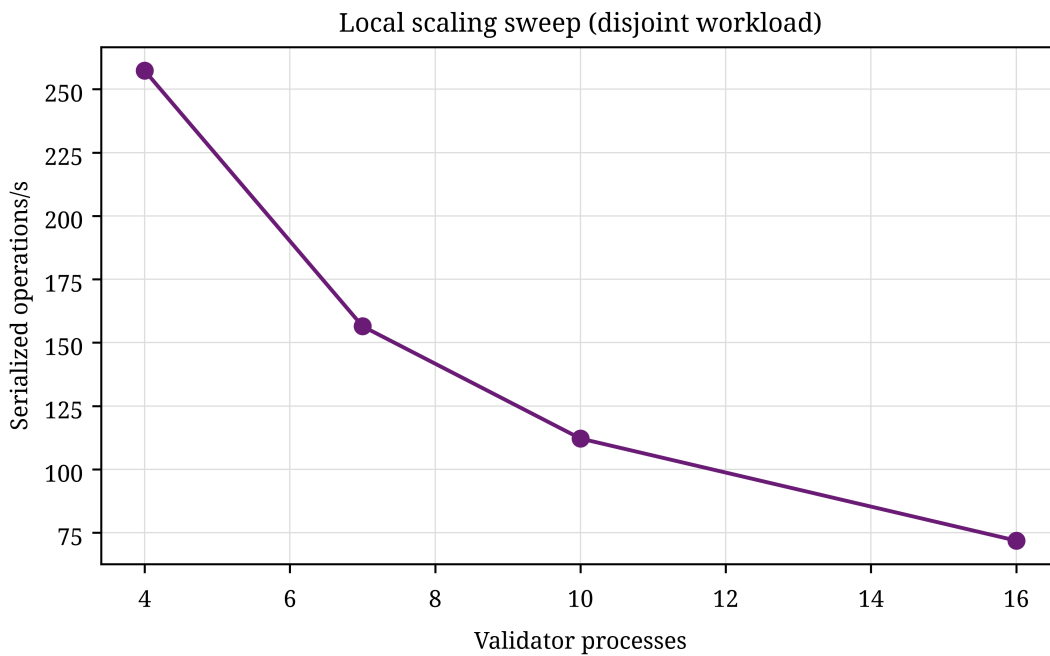


Fig. 7. Serialized local scaling sweep for disjoint transitions. Increasing validator count increases receipt and closure fan-out; the result is not WAN throughput evidence.

bytes.

Table 5. Byte decomposition for the 1,000-operation disjoint workload.

Component	Total bytes	Bytes/op
Capsule wire	514,890	514.89
WitnessReceipt wire	1,304,000	1,304.00
ClosureProof wrapper	317,000	317.00
Closure wire including re-receipts	1,621,000	1,621.00
Successor StateSeal wire	199,780	199.78
Four-byte frame overhead	28,000	28.00
Hash-chained event log	707,548	707.55
ConflictEvidence	0	0.00
Availability	0	0.00
Bundle wrapper	0	0.00
Non-overlapping protocol total	3,071,218	3,071.22

This decomposition explains why an aggregate daemon byte counter can be much larger than a Capsule alone. The present implementation pays for verbose JSON, four receipts per closure, full cryptographic signatures, and a hash-chained audit event. These are implementation costs as well as protocol evidence costs. In the competing-claims workload, ConflictEvidence adds 635 bytes per rejected claim on average. The distinction matters: a binary codec or receipt aggregation could reduce framing and representation overhead without changing the ECTC semantics, whereas removing the witness and closure artifacts would change the protocol claim.

7.5. Long-run recovery rehearsal—The seven-process v5 local rehearsal used two Byzantine identities, one scheduled kill/restart, 120 requested operations, and 24 partial-frame connections. All 120 operations succeeded, the liveness ratio was 1.0, and unexpected operational errors were zero. Median latency was 36.804089 ms and p95 latency was 233.559733 ms. The hash-chained event log contained eight events and verified successfully. Its aggregate daemon counter reported 184,916.02 sent/received bytes per successful operation; this includes node-to-node traffic across the seven processes and is not the same accounting domain as the 3,071.22 in-process non-overlapping protocol bytes per operation in Table 3. The final artifact also records sent and received totals by message type (CAPSULE, CLOSURE, RECEIPT, SUBMIT, and METRICS), making the network counter auditable rather than treating it as an in-process protocol-byte measurement. The previous v3 artifact had recorded CapsuleInvalid: unknown predecessor seal after restart and p95 8,447.689336 ms. The corrected path classifies the predecessor race as pending/protocol rejection and routes early retries to a healthy validator. This is a recovery-path correction, not a general speedup claim.

8. Related Work

8.1. Ordered BFT and DAG dissemination—Practical Byzantine fault tolerance established the classic state-machine replication baseline for signed requests, quorum agreement, and deterministic execution;¹⁰ later systems work emphasizes the composability and implementation boundaries among clients, agreement, execution, checkpointing, and recovery.¹¹ HotStuff is a

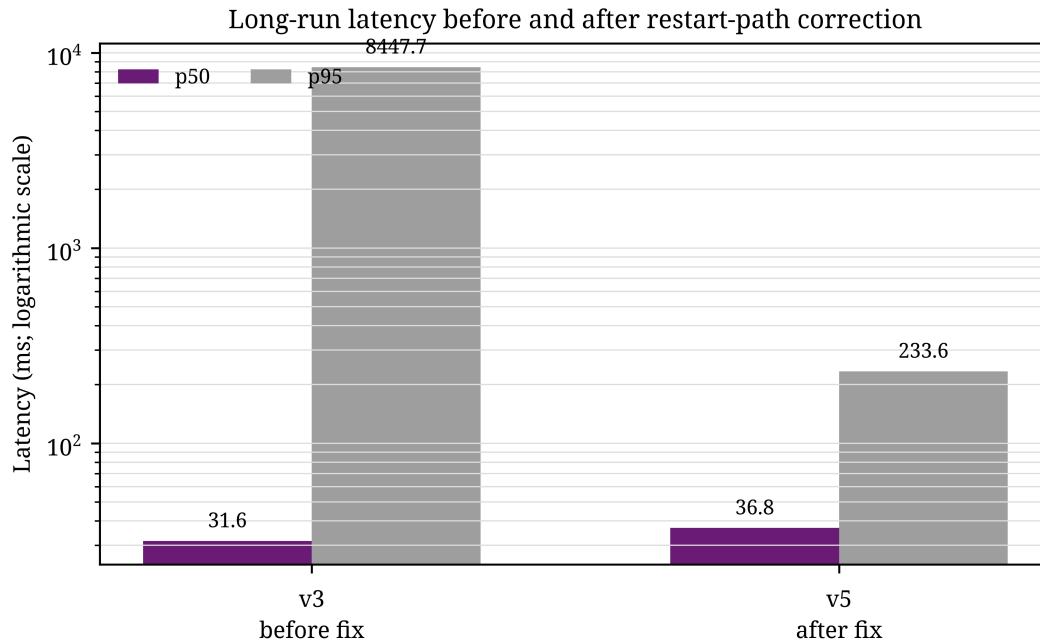


Fig. 8. p50 and p95 latency before and after the restart-path correction in the local long-run artifacts. The logarithmic scale exposes the former recovery tail.

leader-based BFT replication protocol with a partially synchronous safety/liveness model and responsive communication phases.² Narwhal and Tusk separate reliable dissemination from ordering and use a DAG-based mempool with BFT consensus.³ Mysticeti studies low-latency DAG-based BFT and reports a large-scale evaluation.¹² MOSAIC does not replace these protocols or claim a lower communication bound. Its proposed object is a transition lifecycle whose evidence is consumed by execution and audit layers.

8.2. *Certificates and restricted ordering*—FastPay demonstrates a high-throughput settlement protocol based on Byzantine consistent broadcast for pre-funded payments.⁴ Sui Lutrism combines a consensusless agreement path with consensus when transactions risk inconsistent concurrent access.⁵ Sui’s object model makes versions, owners, and authenticated object references first-class.¹³ Polygraph studies accountability in Byzantine agreement by exposing evidence when disagreement becomes unavoidable.¹⁴ These works are close prior art for selective ordering, local certificates, and accountability. MOSAIC’s narrower hypothesis is that the accepted and rejected transition claims should share one explicit evidence-complete closure lifecycle.

8.3. *Object-centric state and parallel execution*—Tango investigates distributed data structures over a shared log and demonstrates that object-level state and versioning can be built above replicated logging.⁶ Block-STM uses optimistic concurrency and deterministic validation to execute transactions in parallel while preserving a preset order.⁷ Process-commutative distributed objects and BFT CRDTs study legal operation traces and the space between conflict-free replication and total-order ledgers.⁸ These results prevent MOSAIC from claiming first use of object-centric execution, optimistic concurrency, commutativity, or reduced ordering scope. Attested append-only memory also shows how authenticated history can constrain an adversary’s later claims.¹⁵ The distinction proposed here is evidence semantics: a transition’s terminal status is Closed only with a closure proof and successor seal, while Conflict and Abandoned

remain independently inspectable artifacts. ECTC is therefore positioned as a specific lifecycle and evidence boundary, not as a claim to invent authenticated state, transaction certificates, or accountability in general.

Table 6. Positioning against adjacent protocol and execution families.

Family	Primary unit	WAN/formal evidence	Relation to MOSAIC
HotStuff	Ordered BFT proposal	Theorem and WAN evaluations	Baseline for quorum safety and fair performance comparison; not replaced by ECTC.
Narwhal/Tusk	Causal DAG plus BFT order	WAN evaluation	Shares explicit dissemination/ordering separation; MOSAIC emphasizes transition evidence.
FastPay	Restricted payment certificate	Protocol evaluation	Close certificate-oriented prior art for a narrower workload.
Sui Lutriss	Object plus selective ordering	Production/WAN evidence	Closest access-aware production comparison; MOSAIC lacks Sui-scale VM and WAN evaluation.
Mysticeti	Uncertified DAG BFT	Theorem and WAN evaluation	Sets a stronger latency/evaluation standard than the local prototype.
Tango	Versioned data structures over shared log	Distributed-system evaluation	Prior art for object-level state and versioning.
Block-STM	Optimistic deterministic execution	Execution benchmarks	Prior art for conflict-aware parallel execution, not closure evidence.
BFT CRDT objects	Commutative operation traces	Formal models	Prior art for reduced ordering and Byzantine objects, not the ECTC lifecycle.
MOSAIC	Capsule and closure evidence	Bounded model/local tests	Hypothesis: every observed terminal transition outcome has a verifiable artifact.

9. Security Analysis

9.1. Threat model—The prototype assumes unforgeable Ed25519 signatures, deterministic canonical serialization, a bounded Byzantine set or stake weight, and honest non-equivocation within the modeled first-claim context. It considers malformed messages, replay, stale predecessors, conflicting Capsules, delayed receipts, validator restart, beacon non-reveal, and local shard loss. It does not claim protection against compromised private keys, compromised hosts, colluding stake majorities, or all network-layer denial-of-service strategies.

9.2. *Safety*—The ECTC invariants and Theorem 1 provide the stated safety boundary. The implementation rejects application without a valid ClosureProof, verifies predecessor and successor bindings, rejects incompatible closures with recorded evidence, and checks execution receipts against successor roots. The TLA+ model and Python checker are bounded/model-checked support, not an unbounded proof of the entire implementation.

9.3. *Liveness*—The local long-run achieved 120/120 completed operations and liveness ratio 1.0 under the declared schedule. The beacon fallback preserves progress when reveal weight is low in the local model. These observations do not establish liveness under arbitrary WAN partitions, operator churn, public admission, or adversarial stake concentration.

9.4. *Accountability*—ConflictEvidence and AbandonProof make negative protocol outcomes inspectable. The hash-chained event log makes operational incidents and upgrades tamper-evident within one log. These artifacts provide evidence for review; they do not by themselves prove that every external observer saw the same message schedule or that a Byzantine operator will accept a slashing decision.

9.5. *Replay and equivocation*—Capsules and receipts bind identifiers, epochs, attempts, predecessor identifiers, signatures, and status. ReplayGuard rejects duplicate message identifiers and the protocol rejects old-epoch receipts. First-claim locks preserve local non-equivocation; quorum intersection supplies the conditional closure-exclusivity argument.

9.6. *Crash recovery*—SQLite WAL, snapshots, and restore paths recover seals, Capsules, receipts, and closures. The long-run correction ensures that a Capsule received before predecessor restoration enters pending/retry handling rather than being counted as an unexpected error. The evidence is local and does not cover correlated storage loss or operator key recovery.

9.7. *Availability*—Reed–Solomon encoding and repair are implemented and tested, and the availability network exposes sampling and repair paths. No claim is made that local shard repair proves a production data-availability layer. A public deployment would need independent providers, retention policies, sampling schedules, withholding tests, and correlated-failure experiments.

9.8. *Permissionless limitations*—MOSAIC is not yet a public permissionless network. The settlement and admission code is an implementation component; permissionless security would require independent operators, Sybil resistance, stake-distribution analysis, enforceable slashing, governance, key isolation, public genesis, and WAN experiments. Economic security, unbiased randomness, and public-token security are not established here.

10. Limitations

The experiments are local and serialized or process-local. The validator scaling sweep increases receipt fan-out but does not model WAN latency, packet loss, heterogeneous hardware, clock drift, NAT, or independent failure domains. The long-run uses seven processes on one host and deterministic Byzantine identities; it is not a public testnet. The execution kernel is intentionally small, the weighted theorem extension is stated under explicit assumptions, and the TLA+ model is bounded. The byte accounting reports JSON and event-log costs, which are implementation-dependent. The paper does not claim that ECTC is globally novel relative to all object-centric, certificate-based, CRDT, or BFT literature; it presents a falsifiable composition and evidence hypothesis.

Table 7. Claim classification.

Claim	Status	Boundary
ECTC lifecycle is executable	Implemented	Reference implementation and focused tests.
Closed requires closure and successor seal	Implemented/tested	ECTC tests and bounded support.
No application without closure	Model-checked/tested	Declared finite model and implementation tests.
No incompatible closures under quorum assumptions	Theorem-supported/model-checked	Conditional argument; not an unbounded executable proof.
Conflict and abandonment evidence	Implemented/tested	Signed evidence objects.
Deterministic execution	Experimentally tested	Supported instruction kernel only.
Crash recovery and malformed traffic	Local observation	One-host rehearsal and network tests.
WAN liveness	Not evaluated	Requires independent hosts and operators.
Permissionless/economic security	Not established	Requires a separate public-network study.
Performance superiority	Not claimed	No apples-to-apples cross-system experiment.

11. Reproducibility

The public repository at <https://github.com/adnanomar77/mosaic-protocol> contains source code, tests, formal files, benchmark drivers, Mermaid sources, rendered figures, CSV data, JSON results, reference-audit output, and checksums. The principal commands are:

```
python3 -m pytest -q
PYTHONPATH=. python3 -m benchmarks.model_check_mosaic
PYTHONPATH=. python3 benchmarks/run_mosaic_ectc_workloads.py --operations 1000
PYTHONPATH=. python3 paper/generate_figures.py
python3 -m benchmarks.run_mosaic_testnet_long --nodes 7 \\\
--operations 120 --base-port 22620 \\\
--data-dir /tmp/mosaic-testnet-long \\\
--event-log testnet/events/testnet-0.jsonl
```

The fixed research artifact is tagged ‘v6.0.0-ectc-paper’ separately from the moving branch. The repository excludes private keys, credentials, local virtual environments, and runtime WAL data. `paper/ledger_submission/checksums.sha256` records the manuscript and submission-package hashes. The values reported in this paper are reproducible workload/artifact values, not a promise of identical microsecond-level timings on another machine.

12. Discussion

The scientific value of MOSAIC, if supported by independent replication, would be a change in the object whose integrity is made explicit. A conventional ledger can retain transaction data, votes, and state roots, but a protocol implementation may still expose a rejected or abandoned claim only as an absence from the committed path. ECTC makes the negative paths part of the claimed lifecycle. That may matter in settlement networks, registries, multi-party workflows, or systems where the reason for rejection is itself subject to audit.

The cost is visible. The 1,000-operation decomposition shows that four signed receipts, closure records, JSON framing, and hash-chained events dominate the measured bytes. Batching

reduces bytes per operation in this implementation, but batching is appropriate only for compatible independent closures. A future binary codec or proof aggregation could reduce representation overhead, but any optimization must preserve the evidence that supports I6 and I7.

The closest future comparison is workload-by-workload rather than “MOSAIC versus blockchain” in the abstract. Disjoint resources should be evaluated for ordering scope and closure cost; simultaneous conflicting resources should be evaluated for detection latency and evidence completeness; shared state should be evaluated for closure contention and recovery; availability should be evaluated for provider overhead and correlated shard loss; and permissionless deployment should be evaluated for admission, stake concentration, slashing, and churn. The current artifacts establish the measurement machinery but do not yet supply the independent-WAN evidence for those claims.

13. Conclusion

This paper introduced MOSAIC and its central semantics, Evidence-Complete Transition Closure. Every observed terminal transition outcome is intended to be Closed with a ClosureProof and successor StateSeal, Conflict with ConflictEvidence, or Abandoned with AbandonProof; pending is explicitly non-terminal. The paper stated the unweighted quorum assumptions for Closure Exclusivity, described a weighted extension, mapped I1–I8 to TLA+, added a 20-case adversarial matrix, and evaluated disjoint, competing-claims, batched, scaled, and crash/restart workloads. The competing-claims experiment closed 1,000 winners, rejected 1,000 losing claims, and generated 1,000 ConflictEvidence artifacts.

The implementation and local artifacts support a coherent protocol hypothesis. They do not establish a public permissionless testnet, WAN liveness, economic security, an unbounded formal proof, or universal performance superiority. Independent WAN deployment with operator-controlled keys, stronger formal checking of epochs and weighted membership, and fair comparisons with HotStuff, Narwhal/Tusk, Sui Lutris, FastPay, Mysticeti, Block-STM, and BFT-object systems are outside the present evaluation scope. Within the reported scope, the defensible claim is that MOSAIC is a reproducible protocol and systems artifact whose distinctive hypothesis is an evidence-complete state-transition closure lifecycle.

Acknowledgements

The author thanks the open-source projects and public research communities whose papers, specifications, and software make comparative protocol research possible. No external funding was received for this work.

Author Contributions

AOAA designed the MOSAIC protocol model, implemented the prototype and benchmark harnesses, ran the experiments, analyzed the artifacts, and prepared the manuscript.

Conflict of Interest

The author declares that he has no known conflicts of interest relevant to this work.

AI Use Disclosure

An AI-assisted software and language workflow was used during project development and manuscript preparation¹⁶ for code inspection, drafting support, organization, and language editing. The author reviewed the implementation, ran the experiments, verified the reported values and sources, and remains solely responsible for the manuscript, its claims, limitations, and final submission. This disclosure follows Ledger’s AI policy.

Data and Code Availability

The source code, benchmark drivers, formal model, generated figures, tabular data, JSON artifacts, adversarial matrix, and checksums are available at <https://github.com/adnanomar77/mosaic-protocol>.¹⁷ The fixed artifact version associated with this revision is identified by repository tag ‘v6.0.0-ectc-paper’ and checksum manifest. The public repository excludes private keys, credentials, and local virtual environments. All current evaluation artifacts retain the LOCAL_EMULATION scope.

Notes and References

¹ Nakamoto, S. “Bitcoin: A Peer-to-Peer Electronic Cash System.” (2008) Technical report URL <https://bitcoin.org/bitcoin.pdf>.

² Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., Abraham, I. “HotStuff: BFT Consensus in the Lens of Blockchain.” *arXiv preprint arXiv:1803.05069* doi:10.48550/arXiv.1803.05069 URL <https://arxiv.org/abs/1803.05069>.

³ Danezis, G., Kokoris Kogias, E., Sonnino, A., Spiegelman, A. “Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus.” In *Proceedings of the Seventeenth European Conference on Computer Systems* 34–50 (2022) doi:10.1145/3492321.3519594 URL <https://arxiv.org/abs/2105.11827>.

⁴ Baudet, M., Danezis, G., Sonnino, A. “FastPay: High-Performance Byzantine Fault Tolerant Settlement.” In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies* 163–177 (2020) doi:10.1145/3419614.3423249 URL <https://arxiv.org/abs/2003.11506>.

⁵ Blackshear, S., *et al.* “Sui Lutriss: A Blockchain Combining Broadcast and Consensus.” *arXiv preprint arXiv:2310.18042* doi:10.48550/arXiv.2310.18042 URL <https://arxiv.org/abs/2310.18042>.

⁶ Balakrishnan, M., *et al.* “Tango: Distributed Data Structures over a Shared Log.” In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* 325–340 (2013) doi:10.1145/2517349.2522732 URL <https://dl.acm.org/doi/abs/10.1145/2517349.2522732>.

⁷ Gelashvili, R., *et al.* “Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing.” *arXiv preprint arXiv:2203.06871* doi:10.48550/arXiv.2203.06871 URL <https://arxiv.org/abs/2203.06871>.

⁸ Frey, D., Guillou, L., Raynal, M., Taiani, F. “Process-Commutative Distributed Objects: From Cryptocurrencies to Byzantine-Fault-Tolerant CRDTs.” *arXiv preprint arXiv:2311.13936* doi:10.48550/arXiv.2311.13936 URL <https://arxiv.org/abs/2311.13936>.

⁹ Reed, I. S., Solomon, G. “Polynomial Codes over Certain Finite Fields.” *Journal of the Society for Industrial and Applied Mathematics* **8.2** 300–304 (1960) doi:10.1137/0108018 URL <https://doi.org/10.1137/0108018>.

¹⁰ Castro, M., Liskov, B. “Practical Byzantine Fault Tolerance.” In *Proceedings of the Third Symposium on Operating Systems Design and Implementation* 173–186 (1999) OSDI’99; ACM bibliographic record 10.5555/296806.296824 URL <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.

- ¹¹ Distler, T. “Byzantine Fault-tolerant State-machine Replication from a Systems Perspective.” *ACM Computing Surveys* **54.1** 1–38 (2021) doi:10.1145/3436728 URL <https://dl.acm.org/doi/10.1145/3436728>.
- ¹² Babel, K., *et al.* “Mysticeti: Reaching the Limits of Latency with Uncertified DAGs.” *arXiv preprint arXiv:2310.14821* doi:10.48550/arXiv.2310.14821 URL <https://arxiv.org/abs/2310.14821>.
- ¹³ Mysten Labs accessed 19 August 2026 “Sui Object Model.” (2026) URL <https://docs.sui.io/develop/sui-architecture/object-model>.
- ¹⁴ Civit, P., Gilbert, S., Gramoli, V. “Polygraph: Accountable Byzantine Agreement.” In *2021 IEEE 41st International Conference on Distributed Computing Systems* 403–413 (2021) doi:10.1109/ICDCS51616.2021.00046 URL <https://ieeexplore.ieee.org/document/9546489>.
- ¹⁵ Chun, B.-G., Maniatis, P., Shenker, S., Kubiawicz, J. “Attested Append-only Memory: Making Adversaries Stick to Their Word.” In *Proceedings of the Twenty-First ACM Symposium on Operating Systems Principles* 189–204 (2007) doi:10.1145/1294261.1294280 URL <https://research.google/pubs/attested-append-only-memory-making-adversaries-stick-to-their-word/>.
- ¹⁶ Ledger Journal accessed 19 August 2026 “Ledger AI Policy.” (2026) URL <https://ledgerjournal.org/ojs/ledger/AI>.
- ¹⁷ Allemon, A. O. A. version associated with the manuscript; accessed 19 August 2026 “MOSAIC: State-Transition-Centric Integrity Capsules for Distributed Ledger Execution.” (2026) Software artifact repository URL <https://github.com/adnanomar77/mosaic-protocol>.

Appendix A: Artifact Manifest

Table 8 lists the principal paths used to reproduce the claims in this paper.

Table 8. Principal artifact paths in the companion repository.

Path	Purpose
<code>mosaic/</code>	Protocol, ECTC outcomes, quorum safety, networking, execution, settlement, beacon, availability, and storage.
<code>tests/</code>	Protocol, ECTC, adversarial, storage, security, execution, and network tests.
<code>benchmarks/run_mosaic_ectc_workloads.py</code>	Disjoint, competing-claims, batching, scaling, conflict-evidence, and byte-accounting benchmark.
<code>benchmarks/run_mosaic_testnet_long.py</code>	Seven-process long-run harness with churn, restart, and event logging.
<code>formal/mosaic_safety.tla</code>	Bounded ECTC safety model and I1–I8 mapping.
<code>formal/ectc_safety_argument.md</code>	Model A theorem and weighted extension argument.
<code>docs/mosaic_adversarial_test_matrix.txt</code>	Twenty-case adversarial validation matrix.
<code>docs/mosaic_ectc_workloads.json</code>	Quantitative workload and overhead results.
<code>paper/generate_figures.py</code>	Deterministic generation of quantitative figures and CSV data.
<code>paper/figures/</code>	Architecture, lifecycle, closure, conflict, recovery, workload, and scaling figures.
<code>paper/ledger_submission/checksums.sha256</code>	Package checksums.